



Recreation of a Phase-Sensitive Dynamic Filter (PSDF)

Python + Spyder IDE - Software-Based Simulation

Course: EEE-3218 — Digital Signal Processing Lab

Year 03 // Semester 02 // Section D1

Based on the 2026 PSDF research paper

Karib Muhammad Shayek (20230105164)

Tasin Hasan Bani (20230105166)

Fazlay Elahi (20220105234)

Jannatul Ferdous Ovy (20230105177)

Phase-Sensitive Dynamic Filters: A Conceptual Model for Energy-Preserving, Instantaneously Tunable, Symbol-Level Signal Processing Using Complex IIR Filters

PSDF = Phase-Sensitive Dynamic Filter

A theoretical model for dynamically tunable filtering

Uses time-varying complex IIR filter coefficients

Can respond to frequency and phase changes at symbol level

Paper studies BFSK, QPSK and interference-suppression cases

Research focus

Fast filter tuning without losing the accumulated signal energy.

Key idea

Use the known signal frequency/phase to change the filter coefficients dynamically.

Our role

Recreate the paper's selected simulation behavior in Python/Spyder.

Why Do We Need Tunable Filters?

- Wireless signals (e.g. frequency-hopped or phase-shift-keyed) constantly change frequency and phase.
- A fixed bandpass filter must be wide enough to cover the whole signal band this lets in extra noise.
- Switched filter banks improve on this, but each switch forces the filter to “resettle,” wasting the start of every symbol.
- The reference paper proposes a filter that can retune instantly without ever needing to resettle.
- Core idea: model the filter as a complex-coefficient IIR (recursive) filter instead of a real-coefficient one.
- Switching only the phase of the coefficients moves the filter's passband — the settling time stays the same because it depends only on the coefficient's magnitude, not its phase.
- Because energy/“charge” is preserved across symbols, the filter never needs to restart — enabling real-time, symbol-level retuning.

What We Are Recreating — and How

- Original paper: theoretical model, validated with MATLAB / Simulink simulations.
- Our project: the same filtering concept, reproduced entirely as a software simulation in Python (Spyder IDE).
- No MATLAB/Simulink and no hardware were used — everything runs as a standalone Python script.
- Goal: reproduce the paper's key behaviors — dynamic bandpass filtering, dynamic notch filtering, and their advantage over fixed/switched filters — for BFSK and QPSK test signals.

The Basic Idea

- A standard recursive (IIR) filter remembers its previous output: new output = a little bit of input + a little bit of the last output.
- Using a complex number for that “memory weight” lets us control two things independently: its size sets the bandwidth / settling time, its angle sets the center frequency.
- To change the passband frequency, we only rotate the angle — the size (and therefore the settling behavior) never changes.
- The same trick works on the input side to lock onto — and later restore — a signal's phase.

Simply put, to gain an advantage in one, you must make a concession (tradeoff) in the other:

Narrow Bandwidth: If you make the filter's bandwidth very small or narrow to perfectly filter the signal and reduce noise, it will take much longer for the filter's output to settle.

Wide Bandwidth: Conversely, if the filter needs to respond very quickly or keep the transition time short, its bandwidth must be kept large, which creates an opportunity for excess noise to enter the system.

WE FIXING THE
Bandwidth-Time Tradeoff

$$\Delta f \cdot \tau_f = \frac{1}{2\pi}$$

Our Main Block Diagram

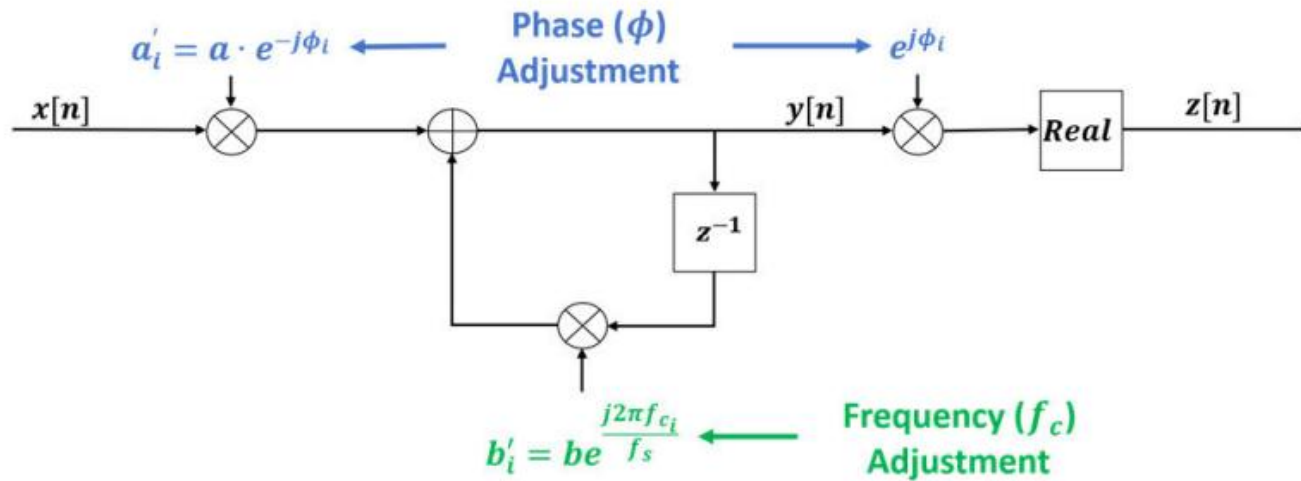
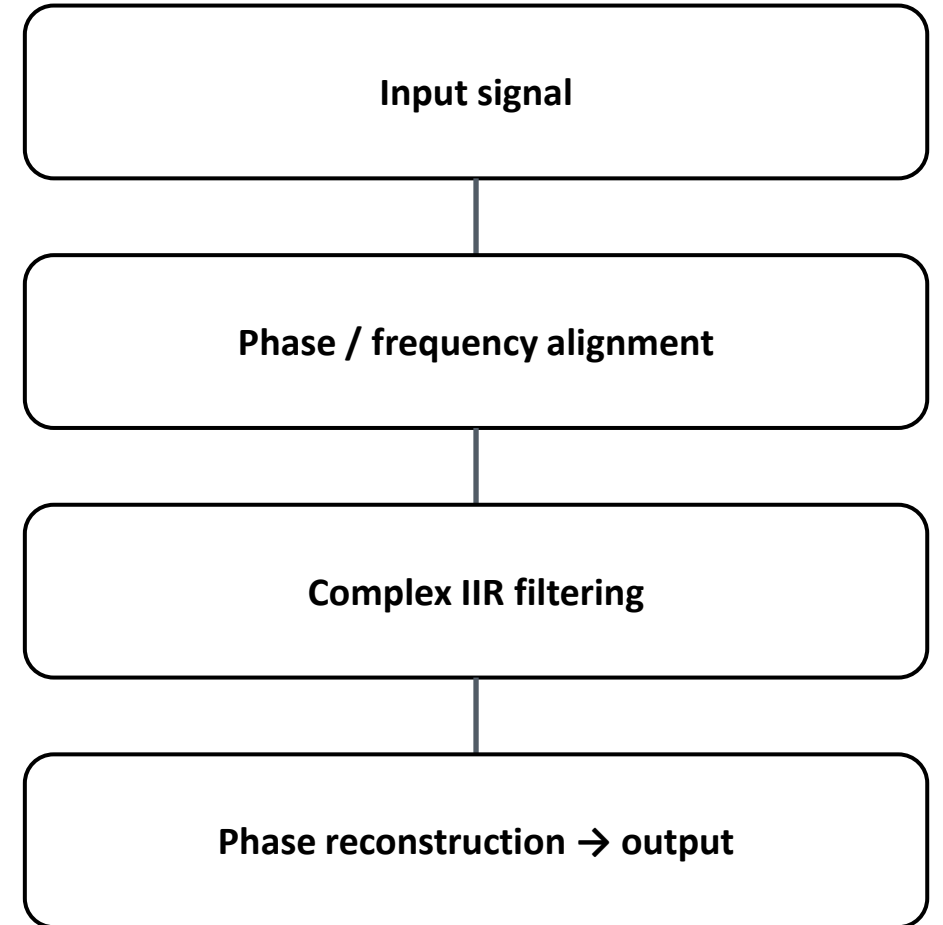
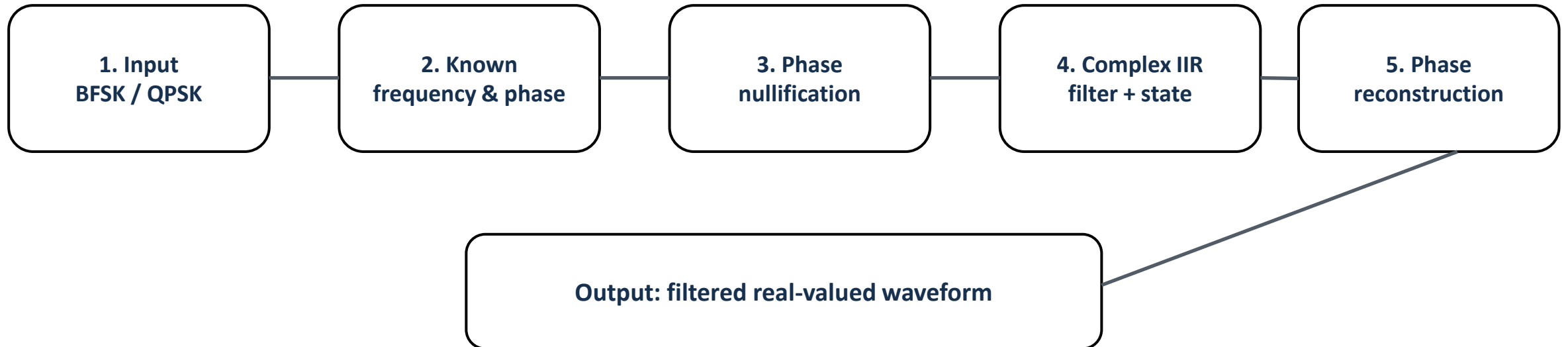


FIGURE 7. Full phase-sensitive dynamic filtering functional block diagram. Adapted from [25].



Simplified View



What Happens Inside Each Block

Dynamic coefficient update → signal-correlated filtering → continuous output

- Input Signal: BFSK toggles between two carrier frequencies; QPSK switches between four phase values — both corrupted with random noise.
- Dynamic PSDF Core: the signal's phase is temporarily removed, passed through a recursive filter whose coefficient angle updates every symbol, then the phase is restored.
- Baseline Filter: the same signal is filtered with fixed (or switched) coefficients as a point of comparison.
- Analysis & Plots: both outputs are compared in the time domain and via frequency spectra to see which one preserves the signal and rejects noise better.

Why Python and Spyder?

- The paper's reference results use MATLAB / Simulink — our team does not have access to a Simulink license, so we rebuilt the model independently in Python.
- Spyder IDE was used for its variable explorer and inline plotting, which made it easy to inspect signals sample-by-sample while debugging.
- Everything is a pure software simulation: NumPy for signal generation and math, SciPy for the Hilbert transform, and Matplotlib for all plots.
- No hardware, no RF front-end — this is strictly a numerical, offline simulation of the filtering concept.
- Python gives direct numerical control over the signal-processing steps
- Spyder provides an easy environment for writing and running the simulation
- NumPy handles arrays and numerical calculations
- Matplotlib creates the time- and frequency-domain plots
- SciPy provides the Hilbert transform used for analytic-signal processing

Reference

MATLAB / Simulink

Our implementation

Python / Spyder

Hardware

None — software simulation only

Python processing Flow

- Signal generation: builds noisy BFSK and QPSK test signals with known symbol timing.
- Filter core: a sample-by-sample loop that updates the filter's complex coefficient at every symbol boundary — this is the heart of the dynamic filtering behavior.
- Baseline comparison filters: fixed-coefficient and switched-coefficient versions, run on the exact same input for a fair comparison.
- Visualization stage: time-domain waveforms and FFT-based spectra plotted with Matplotlib in a consistent, IEEE-style format.

BFSK: frequency changes between two carriers
QPSK: phase changes among four phase states
Recursive states are updated sample-by-sample
FFT plots show the frequency-domain behavior



BFSK — Dynamic Bandpass Filtering

Figure 13: Time-Domain Comparison for BFSK Signals

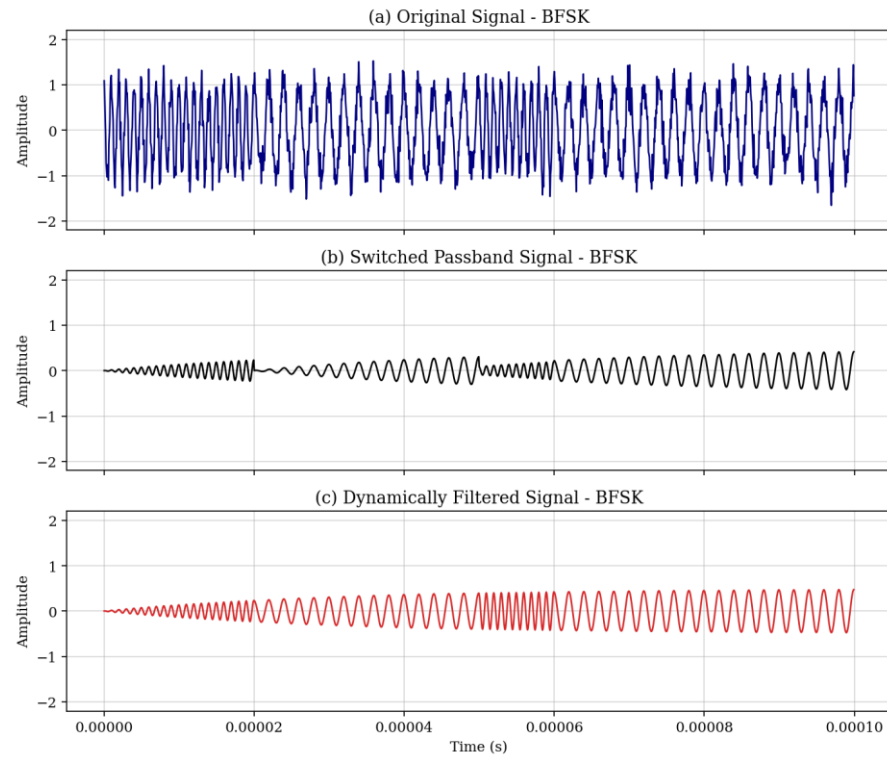
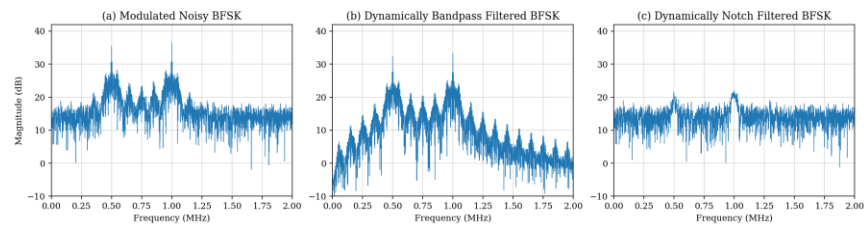


Figure 15: Frequency Spectrum of the BFSK Signal



QPSK — Phase-Sensitive Filtering

Figure 16: Time-Domain Comparison for QPSK Bandpass

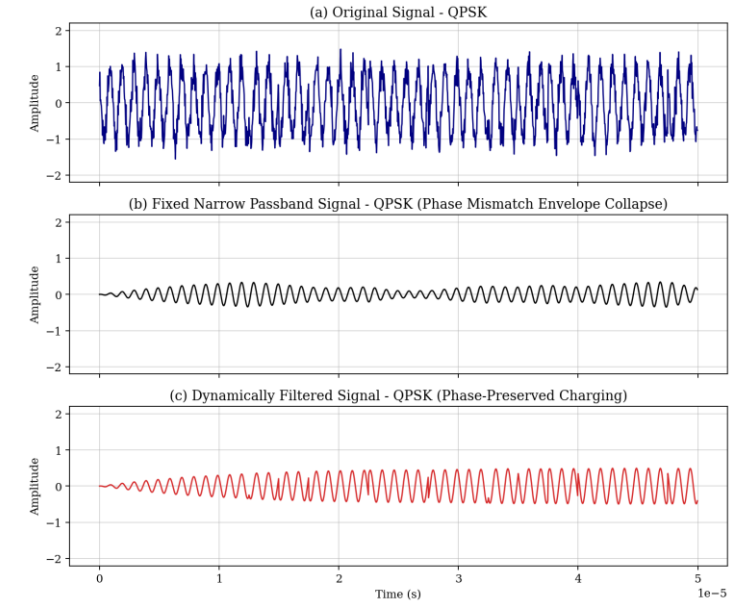
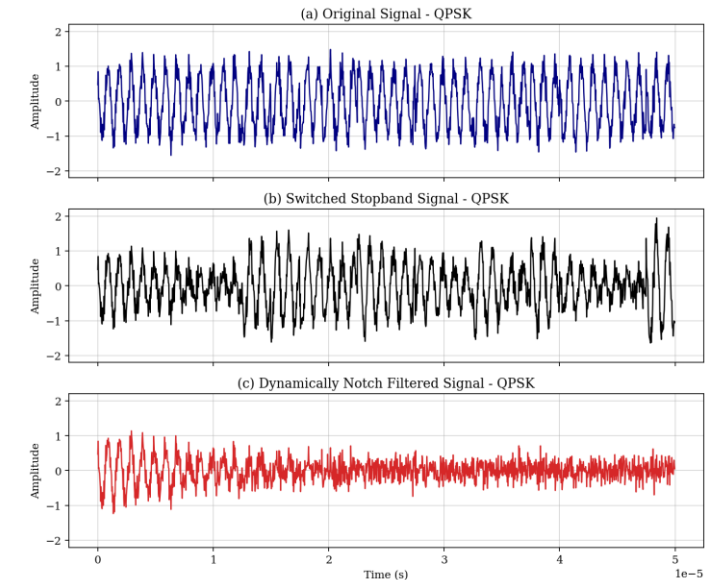


Figure 17: Time-Domain Comparison for QPSK Notch Filtering



What We Tested

- BFSK test: two carrier frequencies, moderate noise, dynamic filter bandwidth set to a fraction of the signal bandwidth.
- QPSK test: four phase states, moderate noise, same comparison approach.
- Extra robustness test: a QPSK signal corrupted by an in-band amplitude-modulated interferer.
- Every test compares three things on the same input: a fixed filter, a switched filter, and our dynamic PSDF filter.

Noise Suppression and Interference Rejection

- Spectrum plots confirm the dynamic filter concentrates energy at the desired carrier while suppressing noise between symbol frequencies.
- In the amplitude-modulated interferer test, the dynamic filter's narrow, phase-locked passband suppressed the interferer while preserving the desired QPSK signal.
- These results are consistent with the qualitative behavior reported in the paper, even though our reference implementation differs (Python vs. MATLAB/Simulink).
- Limitation: our simulation is idealized software only — no hardware noise, component tolerances, or RF non-idealities were modeled.

What we reproduced

- Complex IIR frequency response
- QPSK phase nullification
- BFSK/QPSK dynamic filtering
- AM interference and leakage examples

Important limitations

- Ideal software model only
- No hardware non-idealities
- Requires known frequency/phase information
- Timing/phase mismatch can reduce suppression

Final takeaway

PSDF behavior can be modeled computationally using dynamic complex IIR filtering, with continuous state across signal changes.

Paper concept

Python recreation

Simulation evidence

Conclusion & Future Work

- We successfully recreated the core Phase-Sensitive Dynamic Filter concept as a pure Python software simulation.
- Our results qualitatively match the paper's key claim: switching only the coefficient phase avoids the “resettling” penalty of traditional filters.
- This was a software-only recreation — no MATLAB/Simulink and no hardware were used at any stage.
- Future work: hardware/RF validation, real-time embedded implementation, and testing with higher-order modulation schemes.

Thank You!